

CNN VISIÓN ARTIFICIAL

Storytelling · Simulador Interactivo de Visión por Computadora
con Redes Neuronales Convolucionales

ALUMNA	Camila Cabral
PROFESOR	Alejandro Luis Bonavita
MATERIA	Modelización de Sistemas de IA
FECHA	23 de junio de 2026

■ Imagen Entrada → Convolución → ReLU → Pooling → Clasificación ■

El Problema de la Ceguera Digital

Para un ser humano, ver es un acto inmediato, inconsciente y biológicamente perfecto. Abrimos los ojos y, sin esfuerzo alguno, reconocemos el contorno de un gato dormido bajo el sol, la geometría de un faro recortado contra el horizonte o el patrón regular de una reja. Nuestro cerebro procesa instantáneamente miles de millones de fotones en milisegundos, traduciéndolos en objetos tridimensionales dotados de significado.

Para una computadora, sin embargo, el mundo físico no existe. Una cámara digital simplemente traduce la realidad a un gigantesco mosaico bidimensional de intensidades lumínicas. Una imagen es, en esencia, una matriz numérica: en escala de grises, cada celda vale entre 0 (negro absoluto) y 255 (blanco máximo). En color, es una colección de tres matrices paralelas superpuestas de canales rojo, verde y azul (RGB).

Pregunta Central del Proyecto:

¿Cómo puede una máquina transformar esa grilla de números fríos en comprensión visual? ¿Cómo aprende a reconocer un gato, un auto, o el trazo de una letra en una imagen? Ese es el viaje que este laboratorio interactivo propone recorrer.

Cuando intentamos que una IA convencional entienda esta matriz conectando cada píxel a una neurona tradicional (Red Completamente Conectada / FCN), chocamos con un problema de escala insondable: si una imagen de 1 megapíxel se conectara directamente, necesitaríamos millones de parámetros por capa. La red no solo sería ineficientemente lenta, sino que perdería el contexto espacial: si el gato se desplaza dos píxeles a la izquierda, toda la estructura numérica se vería alterada, causando un fallo total de reconocimiento.

Aquí es donde entran las Redes Neuronales Convolucionales (CNN). Diseñadas a finales de la década de 1980 e inspiradas en la organización biológica de la corteza visual humana, las CNN cambiaron para siempre la forma en que el software interpreta los datos visuales.

Los Tres Pilares de las CNN

Las CNN resuelven el problema de la visión computacional gracias a tres principios fundamentales que las diferencian radicalmente de las redes tradicionales:

01 Invarianza Espacial (Localidad)

En las imágenes de la vida real, la información relevante es local. Un ojo está rodeado de pestañas y párpados; no encontramos la mitad de un ojo en la esquina superior izquierda y la otra mitad en la inferior derecha. Las CNN explotan este principio analizando la imagen a través de ventanas microscópicas llamadas kernels o filtros de 3×3 . En lugar de mirar la imagen completa de golpe, la IA examina pequeñas regiones adyacentes para detectar rasgos locales.

02 Compartición de Parámetros (Eficiencia)

En una red neuronal ordinaria, cada conexión tiene su propio peso numérico exclusivo. En una CNN, un mismo filtro diseñado para detectar líneas diagonales se desliza de forma idéntica a lo largo de toda la imagen. Si el filtro encuentra una diagonal en el centro o en una esquina, se activará con la misma fuerza. Esto reduce la cantidad de parámetros de millones a unos pocos cientos de coeficientes matemáticos repetitivos, logrando un entrenamiento eficiente y rápido.

03 Jerarquía de Características

La corteza visual humana no reconoce un 'gato' de forma instantánea; primero reacciona a bordes de luz simples y luego sintetiza esos bordes en texturas, siluetas e ideas tridimensionales. Las CNN emulan esta jerarquía a través del apilamiento de capas consecutivas.

Capas iniciales (Bajo Nivel): detectan transiciones rápidas de contraste, líneas rectas, diagonales, brillos y sombras.

Capas intermedias (Medio Nivel): combinan las líneas detectadas para reconocer contornos cerrados, elipses y texturas complejas.

Capas profundas (Alto Nivel): ensamblan los contornos para explicar partes reconocibles (ojos, orejas, ruedas) y finalmente clasifican el objeto completo.

Flujo Completo CNN: Imagen Entrada → Convolución + Filtros → Activación ReLU → Pooling Compresión → Clasificación Salida

El Viaje del Píxel: Anatomía Matemática

Etapas 1 — La Convolución y los Filtros Matemáticos

La operación principal consiste en deslizar un Kernel (matriz de 3×3) sobre la imagen de entrada. Al colocarse sobre un parche, calcula el producto escalar local. El píxel de salida resultante en el Feature Map se obtiene multiplicando elemento a elemento (producto de Hadamard) y sumando los productos, más un sesgo opcional:

$$Salida = (\sum_{i,j} I[i,j] \times K[i,j]) + bias$$

Donde **I** es la submatriz de la imagen y **K** es el kernel. El simulador incluye los siguientes filtros con sus propósitos y analogías:

FILTRO	PROPÓSITO	ANALOGÍA
Sobel X (Bordes Verticales)	Detecta cambios de contraste izq → der	Luz rasante del amanecer, muros contra sombras
Bordes Horizontales	Detecta transiciones verticales (grad. Y)	Persiana entreabierta acentuando contornos extremos
Laplaciano	Resalta bordes en todas las direcciones	Dibujante que recorre grietas
Enfoque (Sharpen)	Potencia centro restando influencia del entorno	Lupa de joyero que realza micro-granos
Desenfoque (Blur)	Promedia vecinos reduciendo ruido	Paisaje a través de vidrio empañado
Relieve (Emboss)	Efecto 3D con sombras asimétricas diagonales	Figuras en cartulina húmeda presionada

Etapas 2 — La Rectificación (Función de Activación ReLU)

La convolución es una operación matemática estrictamente lineal. Si solo apiláramos convoluciones tras convoluciones, la red no podría modelar comportamientos complejos y curvos de la vida real —se reduciría a una única operación lineal gigantesca—. Para romper esta linealidad, aplicamos la función ReLU (Rectified Linear Unit) sobre cada píxel de salida:

$$f(x) = \max(0, x)$$

Esto actúa como una compuerta biológica: si la convolución produce un número positivo (el filtro 'encontró' el rasgo buscado), el valor pasa sin alteración. Si produce un número negativo (no hay coincidencia o se obtuvo ruido opuesto), el valor se trunca instantáneamente a cero —el 'silencio neuronal'—. Los contornos descubiertos se iluminan con nitidez sobre un fondo completamente oscuro, listos para la siguiente etapa.

¿Por qué es fundamental?: Sin ReLU u otras no-linealidades, toda la red se reduciría algebraicamente a una sola transformación lineal plana, imposibilitando el reconocimiento de curvas, esquinas y formas complejas del mundo real.

Etapas 3 — La Compresión Espacial (Pooling / Submuestreo)

Para dotar a la IA de invarianza a la traslación y para reducir el coste computacional del entrenamiento, introducimos las capas de Pooling. En la versión más común, Max Pooling 2×2 con stride 2, dividimos el mapa de características en bloques de 2×2 píxeles no superpuestos y conservamos únicamente el valor máximo de cada bloque:

$$Max-Pool([[12, 25], [80, 42]]) \rightarrow 80$$

Esto disminuye la resolución espacial a la mitad —reduciendo la cantidad de datos en un 75%— mientras conserva la intensidad del rasgo estructural más fuerte detectado en esa región. Average Pooling calcula el promedio, preservando información más distribuida pero con menor contraste estructural.

Diseño del Storytelling Interactivo

El eje narrativo del proyecto no es simplemente clasificar imágenes, sino explicar visualmente cómo aprende una CNN. Este enfoque prioriza la enseñanza mediante storytelling, visualizaciones e interactividad, con el objetivo de que una persona sin conocimientos previos comprenda cómo una CNN aprende a interpretar imágenes.

Estructura Narrativa — Las 9 Etapas del Viaje

Nº	ETAPA	DESCRIPCIÓN
1	¿Cómo vemos nosotros?	Mostrar un gato y explicar que una computadora inicialmente solo ve una matriz de números. La imagen como abstracción matemática, punto de partida de todo el aprendizaje.
2	De números a formas	Introducir la convolución mediante filtros para detectar bordes, líneas y esquinas. El primer nivel de abstracción: de píxeles crudos a rasgos locales.
3	La primera capa	Recorrido interactivo por Convolución → ReLU → Pooling con sliders de stride, padding y bias.
4	¿Qué aprendió?	Mostrar mapas de activación (feature maps) de distintos filtros en paralelo en el caleidoscopio de canales.
5	Más capas, más comprensión	Jerarquía de abstracción: bordes simples → texturas → partes de objetos → objeto completo.
6	Pooling comparativo	Comparar la imagen antes y después de Max Pooling y Average Pooling, modificando tamaño de ventana y stride.
7	Laboratorio de hiperparámetros	Sliders para filtros, capas, kernel, padding, stride, learning rate, epochs y batch size con curvas de loss/accuracy en tiempo real.
8	¿Qué está mirando la CNN?	Mapa de calor Grad-CAM simplificado que resalta las zonas de mayor impacto en la decisión del modelo.
9	¿Y si la engañamos?	Probar imágenes borrosas, con ruido gaussiano o parcialmente ocluidas para explicar las limitaciones reales del modelo.

Funcionalidades Extras Interactivas

- Lienzo 16×16 editable píxel a píxel con pincel de precisión y controles de brillo/contraste.
- Monitor de Resonancia Neuronal en tiempo real: activaciones normalizadas por filtro con diagnóstico automático.
- Lupa matemática: desglose paso a paso de cada multiplicación de la convolución en el punto seleccionado.
- Simulador de curvas de entrenamiento (loss / accuracy) con todos los hiperparámetros configurables.
- Quiz de integración: 5 preguntas con retroalimentación educativa inmediata y puntuación final.

Descripción de la Implementación

Stack Tecnológico

COMPONENTE	TECNOLOGÍA / DETALLES
Framework Web	Streamlit ≥ 1.30 — interfaz reactiva sin código HTML/JS adicional
Cómputo Numérico	NumPy — implementación manual de convolución, ReLU y pooling
Visualización	Plotly Express / Graph Objects — gráficos interactivos de feature maps
Tablas / Estado	Pandas — manejo de datos tabulares y estado de sesión
Procesamiento imagen	Pillow (PIL) — soporte para lectura y manipulación de imágenes
Lenguaje	Python 3.10 o superior

Arquitectura del Código

El proyecto se divide en dos módulos. **data.py** centraliza las plantillas de imágenes procedurales en 16×16 , el motor de interpolación bilineal que escala a 64×64 , y el diccionario de kernels con sus matrices de pesos, descripciones y analogías didácticas.

laboratorio_cnn.py contiene el motor matemático (convolución 2D, ReLU, max/avg pooling, ruido gaussiano, blur, oclusión, Grad-CAM) y la interfaz organizada en siete pestañas temáticas: Lienzo Interactivo, Viaje del Píxel, Galería de Filtros, Lupa y Microscopio, Simulador de Entrenamiento, Robustez y Grad-CAM, y Trivia de Integración.

Decisiones de Diseño Destacadas

Implementación manual de CNN: Se optó por implementar convolución, ReLU y pooling desde cero con NumPy en lugar de usar librerías de alto nivel (Keras / PyTorch). Esta decisión es pedagógicamente fundamental: el estudiante puede inspeccionar y modificar cada operación matemática directamente.

Interpolación bilineal $16 \rightarrow 64$: Las imágenes se definen en resolución 16×16 para permitir edición directa píxel a píxel, luego se escalan con interpolación bilineal a 64×64 para visualización fluida, reflejando la idea de reconstrucción continua desde datos discretos.

Grad-CAM simplificado: El mapa de atención se genera como una distribución gaussiana centrada en el focal point de cada plantilla, perturbada por las distorsiones aplicadas. Permite demostrar conceptualmente cómo el ruido y la oclusión degradan el foco atencional del modelo sin requerir gradientes reales.

Diseño visual cyberneuronal: La paleta oscura con acentos en cyan (#00F2FE) y púrpura (#9D4EDD), las fuentes Orbitron y Fira Code, y las tarjetas con gradientes buscan comunicar la naturaleza tecnológica del proyecto reforzando la inmersión del usuario en el entorno de laboratorio.

Experimentos y Resultados Obtenidos

A lo largo del desarrollo del laboratorio se verificaron empíricamente los siguientes comportamientos de la arquitectura CNN:

Bordes Verticales vs Horizontales: Una imagen con líneas verticales dominantes produce alta activación del filtro Sobel X y baja activación del filtro Sobel Y; con líneas horizontales, el comportamiento es inverso. Validado en tiempo real mediante el Monitor de Resonancia Neuronal.

Efecto del Stride en el Feature Map: Con stride=1 el feature map conserva casi toda la resolución espacial. Con stride=3 se comprime agresivamente siguiendo la fórmula: $\text{out} = \text{floor}((\text{in} - k + 2p) / s) + 1$.

ReLU y umbral de activación: Al aumentar el umbral de truncamiento de 0 a +30, se suprimen activaciones de baja magnitud resultando en feature maps más escasos pero con rasgos de mayor contraste relativo.

Degradación por ruido y oclusión: Con ruido gaussiano $\sigma=50$, el mapa Grad-CAM se expande y pierde foco. Con oclusión >40%, el punto focal se desplaza fuera del objeto de interés, demostrando que el modelo debe aprender robustez para ambientes reales.

Curvas de entrenamiento: Learning rate 0.1 produce convergencia inestable. LR 0.001 logra convergencia suave y estable. Epochs > 25 con filtros ≥ 24 activan la señal de sobreajuste. Batch size mayor aporta mayor estabilidad del gradiente y menor tiempo de entrenamiento estimado.

Reflexión Crítica

Logros del Proyecto

- El laboratorio cumple su objetivo central: una persona sin conocimientos previos puede interactuar con cada etapa matemática de una CNN de manera intuitiva y guiada.
- La implementación manual de las operaciones numéricas (sin frameworks de alto nivel) resulta pedagógicamente valiosa al exponer la matemática subyacente de forma transparente.
- El diseño visual de alta fidelidad refuerza la inmersión y el engagement del usuario.
- La integración de Grad-CAM simplificado permite explicar explicabilidad de IA sin requerir gradientes reales, manteniendo la accesibilidad didáctica del proyecto.
- El Quiz de integración al final del flujo consolida el aprendizaje con retroalimentación inmediata y explicaciones contextualizadas.

Áreas de Mejora y Próximos Pasos

- Incorporar un modelo CNN real entrenado con TensorFlow/Keras para mostrar feature maps reales de una red en producción, complementando la simulación matemática.
- Añadir un Constructor Visual de CNN: drag-and-drop de capas para armar y comparar arquitecturas personalizadas.
- Implementar la comparación entre CNN pequeña, mediana y grande en términos de parámetros, tiempo de inferencia y precisión estimada.
- Explorar la conexión con un dataset real (MNIST, CIFAR-10) para que el usuario cargue imágenes propias y vea el pipeline completo.
- Optimizar el rendimiento de las convoluciones con `scipy.signal.convolve2d` o implementación con strides de NumPy para reducir latencia en imágenes de mayor resolución.

Referencias Bibliográficas

Las siguientes tres fuentes cubren el recorrido completo del proyecto: cómo procesa una imagen, qué aprende internamente, y cómo podemos visualizar e interpretar sus decisiones.

Feature Visualization

Olah, C., Mordvintsev, A., & Schubert, L. (2017). Feature Visualization. Distill.

Relevancia: Cubre qué aprende internamente una CNN: visualización de los patrones que maximizan la activación de cada neurona en cada capa.

A Deeper Dive into How CNNs Interpret Images

DigitalOcean Community Tutorial.

Relevancia: Cubre cómo procesa una CNN una imagen: explicación detallada del flujo convolución → activación → pooling con ejemplos visuales de feature maps.

Grad-CAM: Visual Explanations from Deep Networks

Selvaraju, R. R. et al. (2017). Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization. ICCV 2017. También disponible en: MATLAB & Simulink Documentation.

Relevancia: Cubre cómo saber por qué el modelo tomó una decisión: mapas de calor que resaltan las regiones de la imagen que más influyeron en la predicción final.

Nota: El proyecto evitó deliberadamente desviarse hacia temas secundarios como DeepDream, ejemplos adversariales o técnicas avanzadas de optimización de activaciones distintas de Grad-CAM, para mantener el foco en el objetivo central: comprender cómo una CNN transforma una imagen en una representación interna y cómo podemos visualizar ese proceso.